Lab – Building a Custom Docker Image for a Node.js Application

To Begin with the Lab

Summary

In this lab, we learn how to build a custom Docker image for a Node.js application using a Dockerfile. We use the official Node.js base image, which provides the Node.js runtime environment required to execute our application. We use the COPY instruction to copy application files into the Docker image, EXPOSE to document the port used by the application, and CMD to specify the command that runs the server.js file. Finally, we build a custom Docker image, create a container, and access the Node.js application through a web browser.

Understanding Node.js Docker Image

A Node.js Docker image is a preconfigured environment that contains the Node.js runtime required to execute JavaScript applications.

Instead of installing Node.js directly on our host machine, we can use the official Node.js Docker image as the base image.

We can then add our application files and configure Docker to execute the application when the container starts.

Why do we use a Node.js base image?

- **Runtime Environment:** Provides the Node.js runtime.
- **Application Deployment:** Allows us to run Node.js applications inside containers.
- **Port Configuration:** Allows us to document the network port used by the application.
- **Automation:** Automatically starts the application using the CMD instruction.
- **Portability:** Runs the application consistently across different environments.

Example:

Suppose we have a Node.js application containing a file named server.js.

We can create a Dockerfile:

```
FROM node:latest


COPY . .


EXPOSE 80
```

CMD ["node", "server.js"]

Lab Steps

Step 1: Create the Project Folder

• Create a new folder named server-app.

• Open the folder in VS Code.

• Create a file named server.js.

Project structure:

server-app/

```
|
└── server.js
```

Step 2: Create the Node.js Application

Inside server.js, add the following code:

```
var http = require("http");
http.createServer(function (request, response) {
response.writeHead(200, {'Content-Type': 'text/plain'});
   response.end('We are running from within a container\n');
   console.log('The server is running from within a container');
 }).listen(80)
```



Explanation:

- require('http') – Imports the Node.js HTTP module.
- http.createServer() – Creates an HTTP server.
- res.writeHead(200) – Sends an HTTP success status.

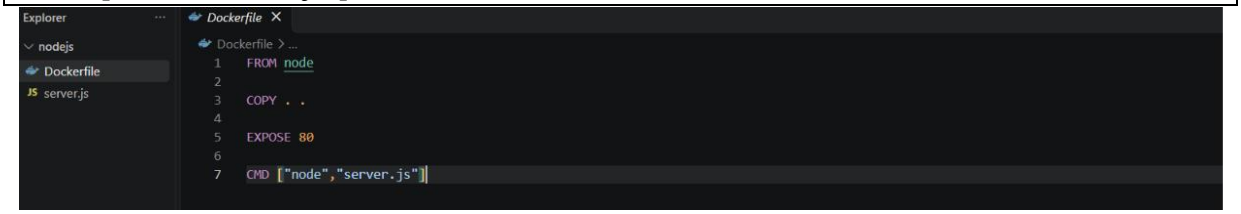
- res.end() – Sends the response to the client.
- server.listen(80) – Starts the server on port 80.

Step 3: Create a Dockerfile

• Create a new file named Dockerfile in the same directory.

• Add the following instructions:

```
FROM node
COPY . .
EXPOSE 80
CMD ["node","server.js"]
```



Understanding the Dockerfile Instructions

1. FROM

FROM node:latest

• Uses the official Node.js image as the base image.

• Provides the Node.js runtime environment.

2. COPY

COPY . .

• Copies all files from the current build context into the working directory of the image.

• In this example, it copies server.js into the image.

3. EXPOSE

EXPOSE 80

• Documents that the application listens on port 80 inside the container.

• Helps communicate the intended network port.

Important: EXPOSE does not actually publish the port to the host machine. We use the -p option with docker run to publish the port.

4. CMD

CMD ["node", "server.js"]

• Specifies the default command to execute when the container starts.

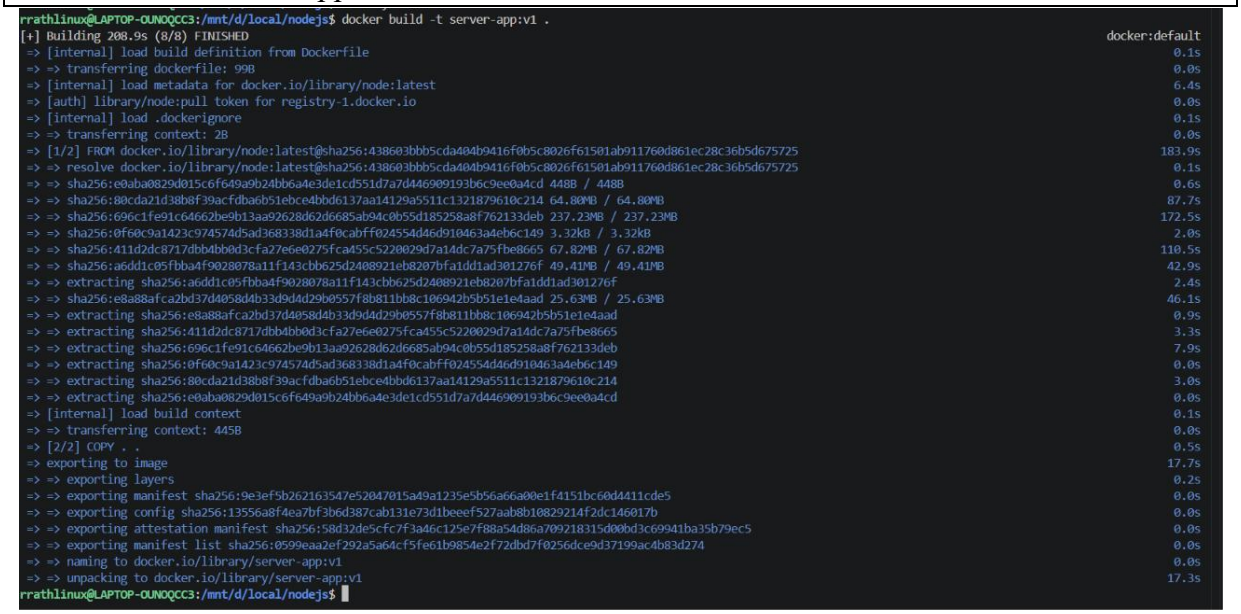
• Runs the server.js file using the Node.js runtime.

Step 4: Build the Docker Image

Open the terminal inside your project directory.

Execute:

```
docker build -t server-app:v1 .
```



Explanation:

- docker build – Builds a Docker image.
- -t – Assigns a name and tag.
- server-app – Image name.
- v1 – Version 1 tag.
- . – Specifies the current directory as the build context.

Docker will:

1. Download the Node.js base image if it is not available locally.
2. Copy the application files.
3. Process the EXPOSE instruction.
4. Configure the CMD instruction.
5. Build the custom Docker image.

Step 5: Verify the Docker Image

Execute:

```
docker images
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$ docker images

IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
server-app:v1  0599eaa2ef29       1.78GB          445MB
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$
```

You should see your newly created image:

REPOSITORY TAG IMAGE ID

server-app v1 xxxxxxxxx

Our custom Node.js Docker image has been created successfully.

Step 6: Check Existing Containers

Before running the new container, check whether another container is already using port 80 on the host.

Execute:

```
docker ps
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$
```

If your previous static application container is running, stop it:

docker stop static-app-instance

Explanation:

• Stops the existing container.

• Frees the host port if that container was publishing port 80.

Step 7: Run a Container from the Node.js Image

Execute:

```
docker run -d -p 80:80 --name server-app-instance server-app:v1
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$ docker run -d -p 8080 --name server-app-instance server-app:v1
677e2d8c337247b6ed166ff7b53a3581e1dacfe53958b15f24cfaf714f5d8883
rrathlinux@LAPTOP-OUNOQCC3:/mnt/d/local/nodejs$
```

Explanation:

Option	Description
docker run	Creates and starts a container.
-d	Runs the container in detached mode.
-p 80:80	Maps host port 80 to container port 80.
--name	Assigns a custom container name.

Option	Description
server-app:v1	Specifies the Docker image and version.

Important: If host port 80 is already occupied, use another host port, such as:

```
docker run -d -p 8080:80 --name server-app-instance server-app:v1
```

Step 8: Verify the Running Container

Execute:

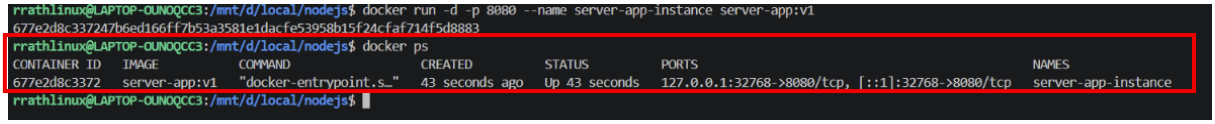
```
docker ps
```

You should see:

CONTAINER ID IMAGE STATUS PORTS

xxxxxxxx server-app:v1 Up 0.0.0.0:80->80/tcp

The container should be running successfully.



Step 9: Access the Node.js Application

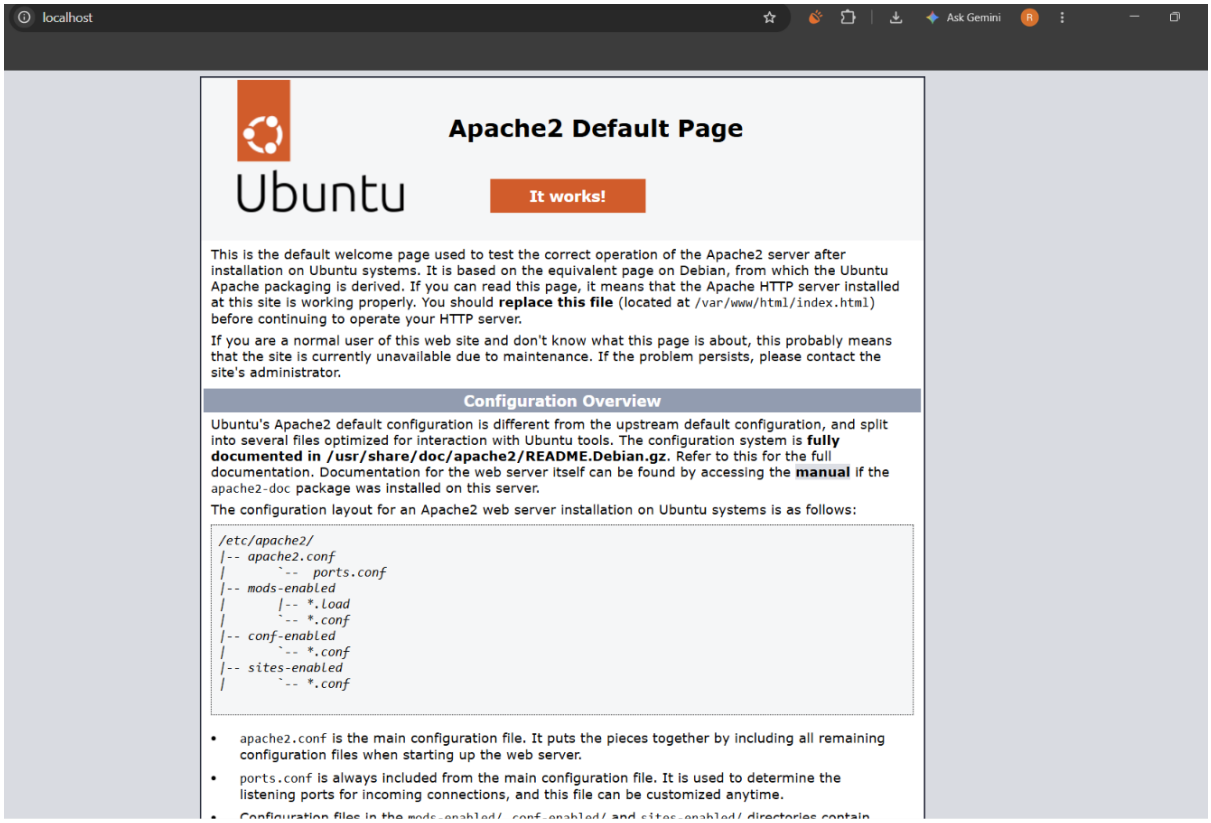
Open your web browser.

Enter:

http://localhost

If you used port 8080, enter:

http://localhost:8080



This confirms that our Node.js application is running inside a Docker container.

Step 10: Check Container Logs

To view the application logs, execute:

```
docker logs server-app-instance
```

Expected output:

Server running on port 80